

[SOEN330679] CÔNG NGHỆ PHẦN MỀM

CHƯƠNG 5: THIẾT KẾ XỬ LÝ (01)

ThS. Võ Quang Phúc

MỤC TIÊU

1. **MT1.** Hiểu được định nghĩa, mục đích và các thành phần chính của lược đồ tương tác trong UML.
2. **MT2.** Đọc và vẽ được Sequence Diagram với đầy đủ cú pháp.
3. **MT3.** Áp dụng được các nguyên tắc vẽ lược đồ tuần tự và nhận diện được các lỗi thường gặp, để tạo ra sequence diagram rõ ràng và dễ chuyển thành code.
4. **MT4.** Phân biệt được lớp dữ liệu và lớp xử lý, biết tổ chức các lớp xử lý theo kiến trúc phân tầng (Presentation, Business, Data Access) để hệ thống dễ bảo trì và mở rộng.

NỘI DUNG

1. **Lược đồ tương tác trong UML**
2. **Ví dụ minh họa**
3. **Nguyên tắc vẽ lược đồ tuần tự**
4. **Lớp xử lý**
5. **SOLID Principles trong lược đồ lớp**
6. **Thực hành**

1. Lược đồ tương tác trong UML

1.1. Định nghĩa

- ❑ Lược đồ tương tác là tập hợp các lược đồ trong UML dùng để mô tả cách các đối tượng (objects) hoặc thành phần (components) trong một hệ thống giao tiếp và cộng tác với nhau nhằm thực hiện một chức năng cụ thể.
- ❑ Lược đồ tương tác tập trung vào khía cạnh động (dynamic behavior) của hệ thống, tức là hành vi theo thời gian, thay vì cấu trúc tĩnh như lược đồ lớp hay ERD.

1. Lược đồ tương tác trong UML

1.1. Định nghĩa

Loại lược đồ	Nhấn mạnh vào	Khi nào dùng
Sequence Diagram (Lược đồ tuần tự)	Thứ tự thời gian của các thông điệp	Mô tả luồng xử lý chi tiết, có nhánh điều kiện, vòng lặp.
Communication Diagram (Lược đồ giao tiếp)	Quan hệ cấu trúc giữa các đối tượng	Thấy nhanh đối tượng nào giao tiếp với đối tượng nào, ít quan tâm thứ tự
Timing Diagram (Lược đồ thời gian)	Trạng thái của đối tượng theo thời gian thực	Hệ thống nhúng, real-time systems, ít dùng trong phần mềm ứng dụng thông thường
Interaction Overview Diagram	Luồng điều khiển tổng thể giữa nhiều interaction	Hệ thống phức tạp cần tổng hợp nhiều Sequence Diagram lại với nhau

1. Lược đồ tương tác trong UML

1.2. Sequence Diagram

- ❑ Sequence Diagram là một loại lược đồ tương tác trong UML mô tả sự trao đổi thông điệp (messages) giữa các đối tượng hoặc thành phần theo một trình tự thời gian tuyến tính từ trên xuống dưới.
- ❑ Mỗi Sequence Diagram đặc tả một kịch bản (scenario) cụ thể của một ca sử dụng (use case), thể hiện ai làm gì, gọi ai, và theo thứ tự như thế nào

1. Lược đồ tương tác trong UML

1.2. Sequence Diagram

Sequence Diagram trả lời ba câu hỏi cốt lõi:

- ❑ Ai tham gia? → Xác định các đối tượng (lifeline) cần thiết để thực hiện chức năng.
- ❑ Làm gì? → Xác định các thông điệp (message) trao đổi giữa các đối tượng.
- ❑ Theo thứ tự nào? → Xác định trình tự thời gian và các điều kiện rẽ nhánh.

1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram

Một Sequence Diagram hoàn chỉnh bao gồm 5 thành phần chính:

- 1) Frame (Khung)
- 2) Lifeline (Đường sống)
- 3) Activation Bar (Thanh kích hoạt)
- 4) Message (Thông điệp)
- 5) Combined Fragment (Đoạn kết hợp)

1. Lược đồ tương tác trong UML

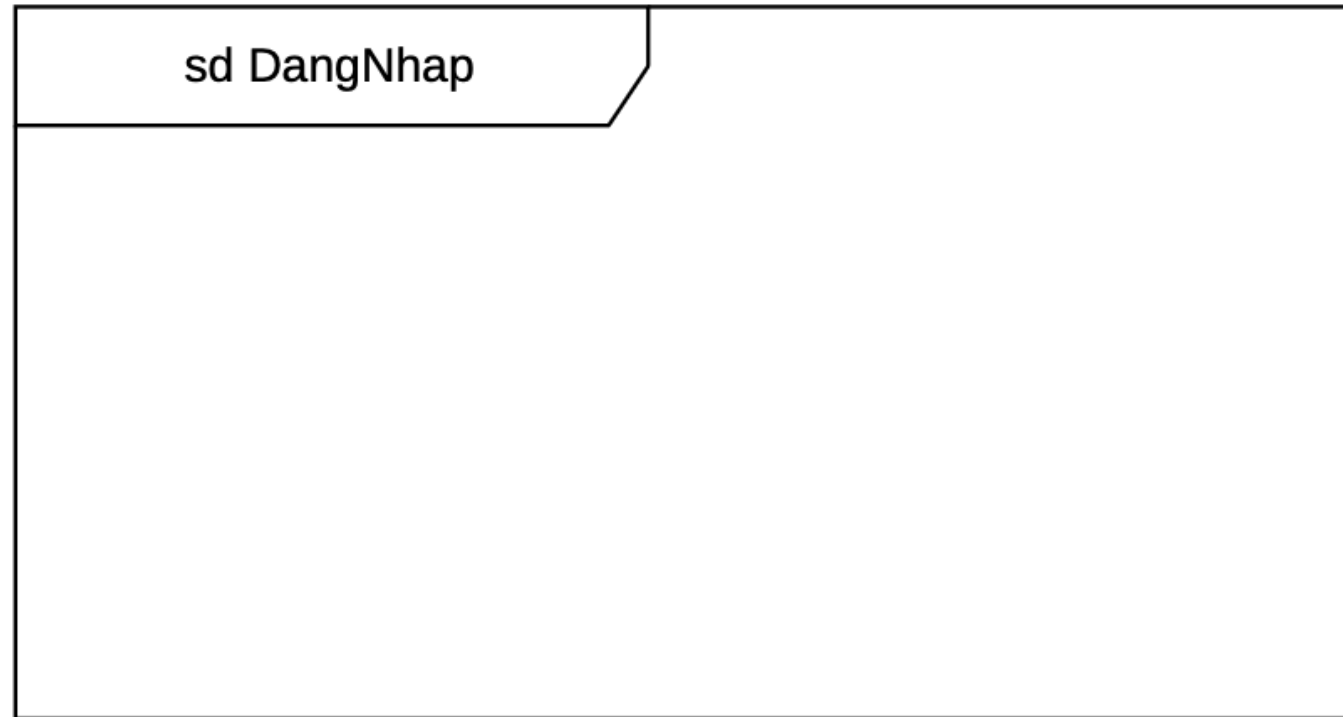
1.2.1. Các thành phần của Sequence Diagram

Frame (Khung)

- ❑ Frame là hình chữ nhật bao quanh toàn bộ Sequence Diagram, có nhãn ở góc trên trái theo cú pháp: **sd TênDiagram**. Phần 'sd' là viết tắt của 'sequence diagram'.
- ❑ Frame xác định phạm vi và tên của toàn bộ tương tác được mô tả.
- ❑ Ví dụ nhãn frame: sd DangNhap, sd DatHang, sd ThanhToan

1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram Frame (Khung)



1. Lược đồ tương tác trong UML

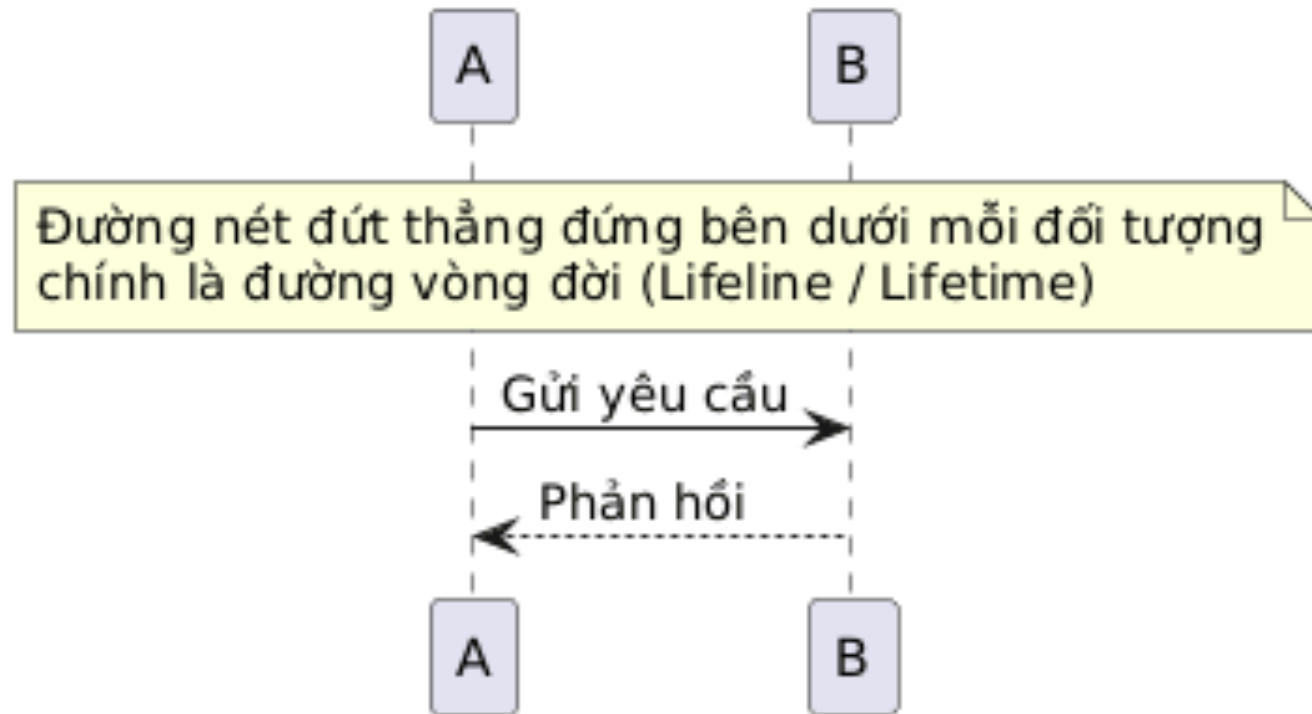
1.2.1. Các thành phần của Sequence Diagram

Lifeline (Đường sống)

- ❑ Lifeline là thành phần đại diện cho một đối tượng, thành phần, hoặc actor tham gia vào tương tác.
- ❑ Lifeline được biểu diễn bằng một hình chữ nhật (head) ở trên cùng, bên trong ghi tên **đối tượng**, và một đường nét đứt thẳng đứng kéo dài xuống phía dưới, đường nét đứt này biểu diễn sự tồn tại của đối tượng trong suốt thời gian tương tác.

1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram Lifeline (Đường sống)



1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram

Đối tượng trong lifeline

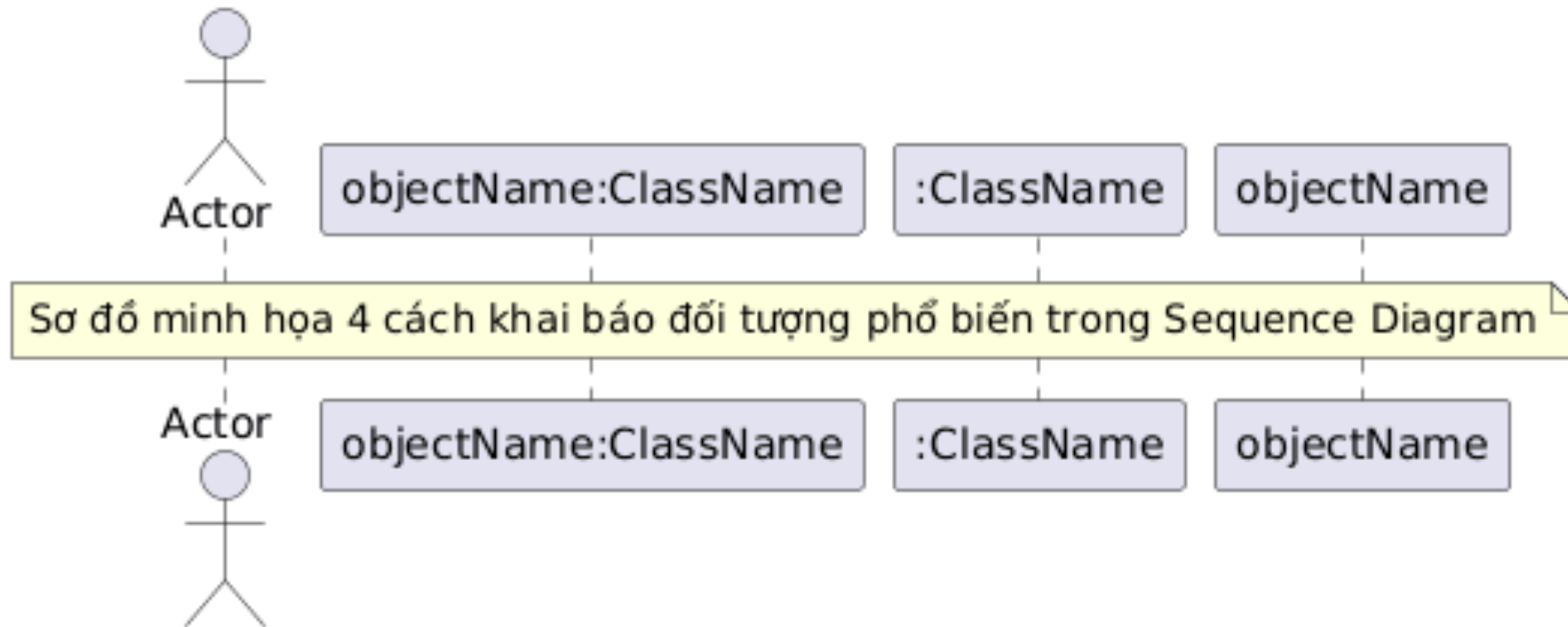
Cú pháp	Ý nghĩa	Ví dụ
objectName:ClassName	Đối tượng cụ thể của một lớp	order:Order, currentUser:User
:ClassName	Đối tượng ẩn danh (chỉ biết lớp)	:OrderService, :UserRepository
objectName	Đối tượng không rõ lớp (ít dùng)	thường tránh dùng vì thiếu thông tin
Actor	Người dùng hoặc hệ thống ngoài	Vẽ bằng hình người, không dùng hình chữ nhật

- Mỗi lifeline đại diện cho một thực thể duy nhất. Nếu hệ thống tạo ra nhiều đối tượng cùng loại trong một tương tác, mỗi đối tượng đó cần một lifeline riêng

1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram

Đối tượng trong lifeline



1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram

Activation Bar (Thanh kích hoạt)

- ❑ Activation Bar là một hình chữ nhật hẹp và dài nằm chồng lên đường lifeline, biểu diễn khoảng thời gian mà một đối tượng đang thực sự thực thi, tức là đang xử lý một thông điệp hoặc đang chờ kết quả từ một lời gọi đồng bộ.
- ❑ Activation Bar bắt đầu khi đối tượng nhận được thông điệp và kết thúc khi đối tượng hoàn thành xử lý và gửi trả kết quả (nếu có).

1. Lược đồ tương tác trong UML

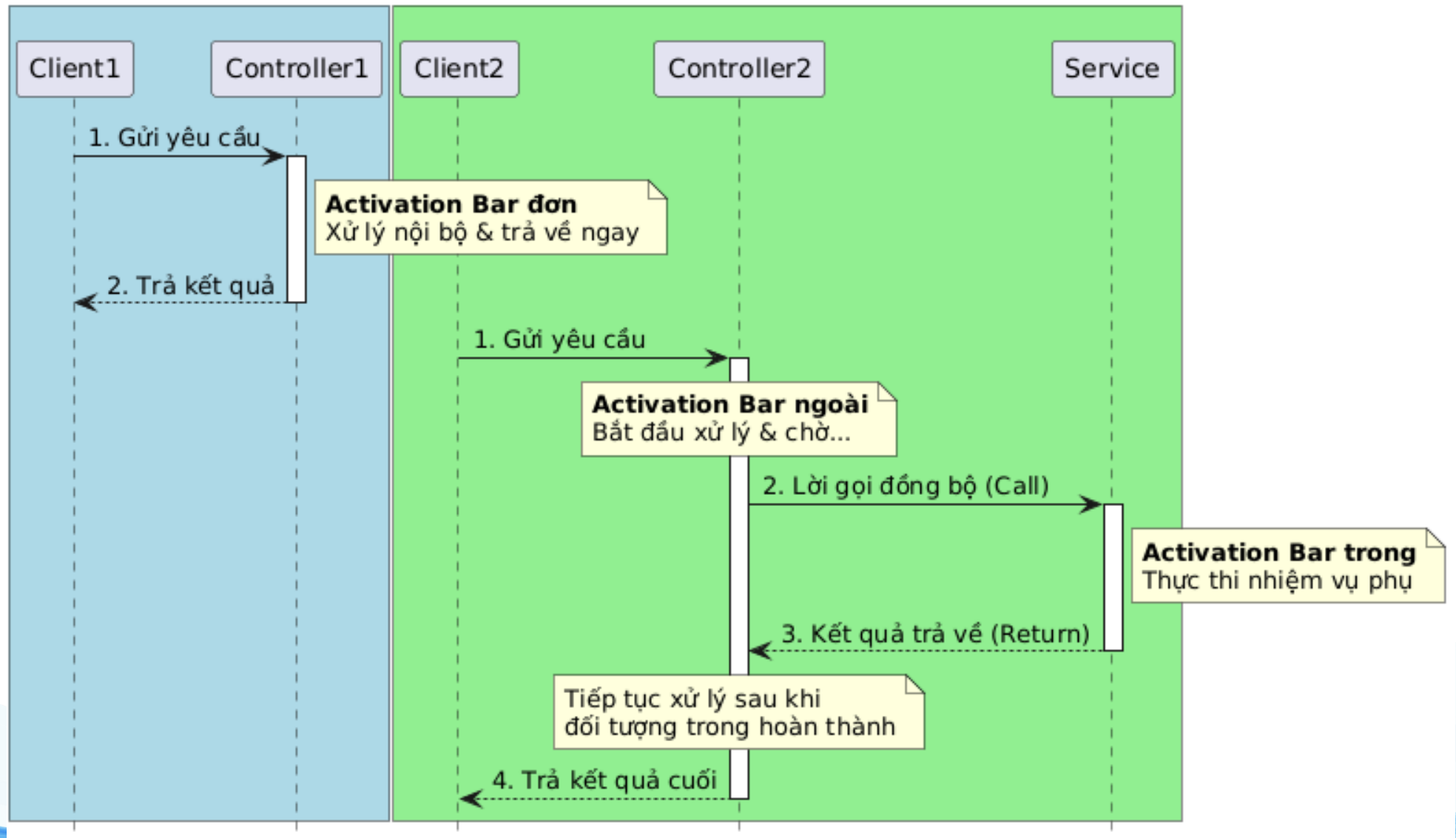
1.2.1. Các thành phần của Sequence Diagram

Activation Bar (Thanh kích hoạt)

Hai trường hợp thường gặp:

- ❑ Activation Bar đơn: đối tượng nhận thông điệp, xử lý và trả về ngay mà không cần gọi đối tượng khác.
- ❑ Activation Bar lồng nhau: đối tượng đang xử lý (activation bar ngoài) gọi đến đối tượng khác (activation bar trong). Đối tượng ngoài sẽ đợi đến khi đối tượng trong hoàn thành, đây là cơ chế của lời gọi đồng bộ (synchronous call).

1. Lược đồ tương tác trong UML



1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram

Message (Thông điệp)

- ❑ Message là đơn vị giao tiếp cơ bản trong Sequence Diagram, biểu diễn sự truyền thông tin (hoặc lời gọi) từ một đối tượng nguồn (sender) đến một đối tượng đích (receiver).
- ❑ Message được vẽ bằng một mũi tên ngang nối từ activation bar của sender đến lifeline của receiver. Nội dung (tên phương thức và tham số) ghi phía trên mũi tên.

1. Lược đồ tương tác trong UML

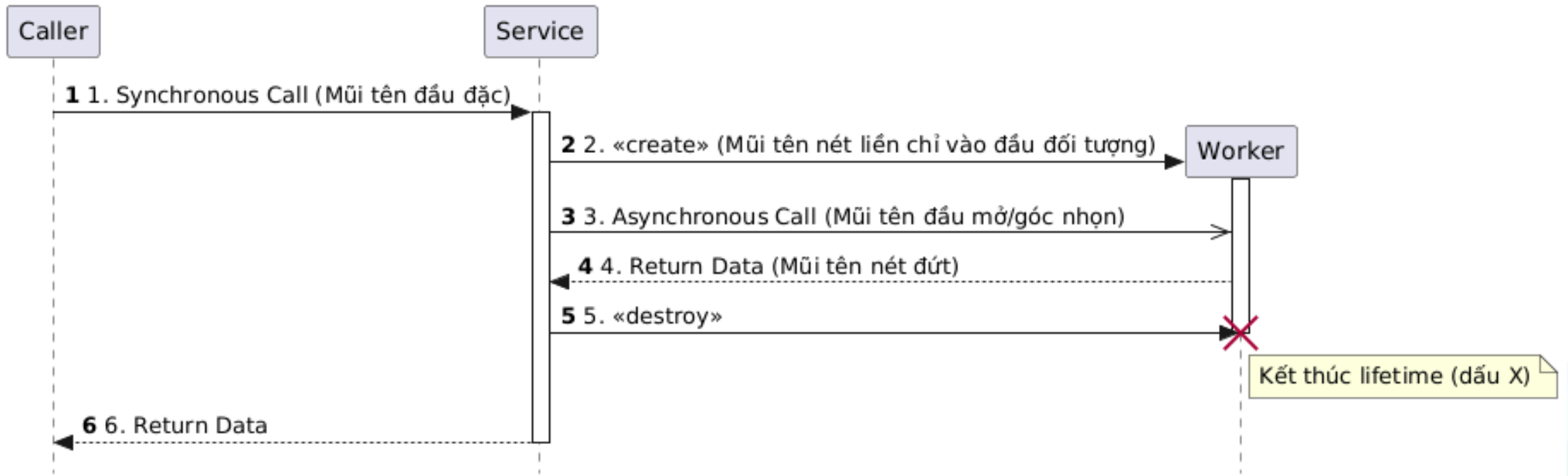
1.2.1. Các thành phần của Sequence Diagram

Message (Thông điệp)

Loại message	Ký hiệu mũi tên	Đặc điểm	Khi nào dùng
Synchronous (đồng bộ)	Đặc → (filled arrowhead)	Sender dừng và CHỜ receiver xử lý xong mới tiếp tục. Tạo ra activation bar lồng nhau.	Lời gọi hàm thông thường, phổ biến nhất trong ứng dụng.
Asynchronous (bất đồng bộ)	Mở → (open arrowhead)	Sender gửi và KHÔNG chờ, tiếp tục xử lý ngay. Không tạo activation bar lồng nhau ở sender.	Event-driven, message queue (Kafka, RabbitMQ), gửi email nền.
Return (trả về)	Đứt ← (dashed, open)	Kết quả trả về từ receiver về sender sau khi xử lý xong. Kết thúc activation bar của receiver.	Sau mỗi synchronous call nên có return message (có thể lược bỏ nếu không có giá trị trả về).
Create (tạo đối tượng)	Liên → đến hộp lifeline	Tạo một đối tượng mới trong quá trình tương tác. Mũi tên trở thẳng đến hộp tên lifeline.	Khi một lớp khởi tạo đối tượng khác: new Order(), new Session().
Destroy (hủy đối tượng)	→ X (kết thúc bằng dấu X)	Hủy một đối tượng. Lifeline kết thúc tại dấu X. Đối tượng không còn tồn tại sau điểm này.	Khi đối tượng tạm thời cần giải phóng: đóng kết nối, xóa session tạm.

1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram Message (Thông điệp)



1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram

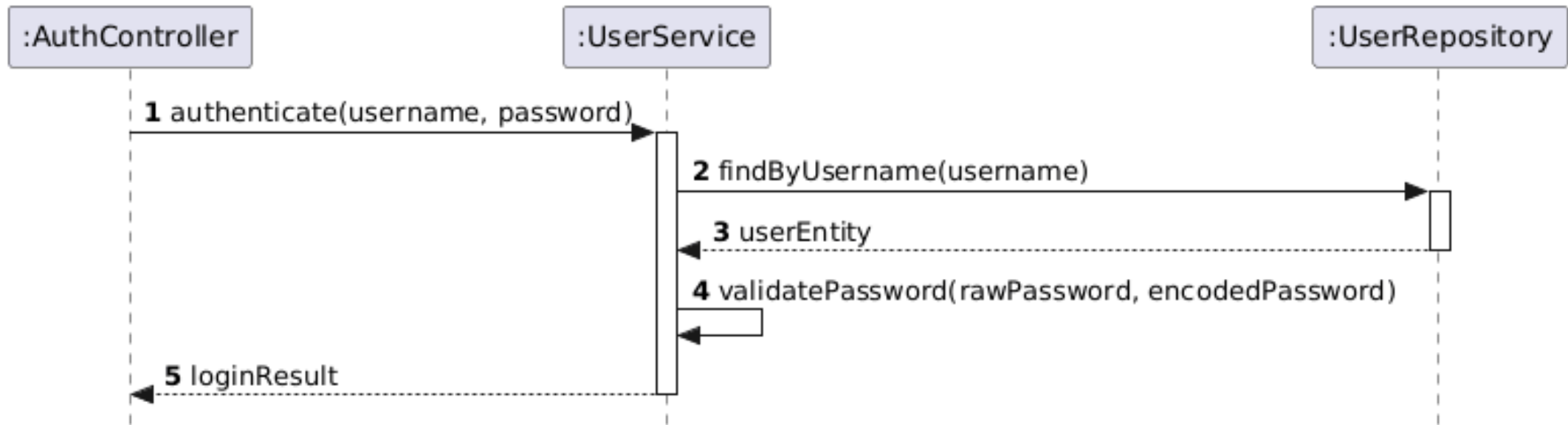
Message (Thông điệp)

Quy tắc đặt tên message:

- ❑ Tên message nên là tên phương thức thực tế, viết theo camelCase, kèm theo tham số nếu cần rõ nghĩa.
- ❑ Ví dụ: `authenticate(username, password)`, `findById(orderId)`, `calculateTotal(items)`.
- ❑ Tránh đặt tên chung chung như 'gửi dữ liệu', 'xử lý' vì không cung cấp đủ thông tin cho lập trình viên.

1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram Message (Thông điệp)



1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram

Combined Fragment (Đoạn kết hợp)

- ❑ Combined Fragment là một cơ chế trong Sequence Diagram dùng để biểu diễn các cấu trúc điều khiển luồng (control flow) như điều kiện, vòng lặp, xử lý ngoại lệ và xử lý song song.
- ❑ Combined Fragment được vẽ bằng một hình chữ nhật lớn bao quanh một phần của diagram, với nhãn (interaction operator) ghi ở góc trên trái bên trong hình chữ nhật, và điều kiện (guard condition) ghi trong dấu ngoặc vuông [].

1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram

Combined Fragment (Đoạn kết hợp)

UML 2.5 định nghĩa 13 loại Combined Fragment, nhưng trong thực tiễn thiết kế phần mềm ứng dụng, 6 loại sau đây là phổ biến và cần thiết nhất:

- 1) alt: Alternatives (Rẽ nhánh điều kiện)
- 2) opt: Option (Tùy chọn)
- 3) loop: Loop (Vòng lặp)
- 4) par: Parallel (Song song)
- 5) ref: Reference (Tham chiếu)
- 6) break: Break (Ngắt)

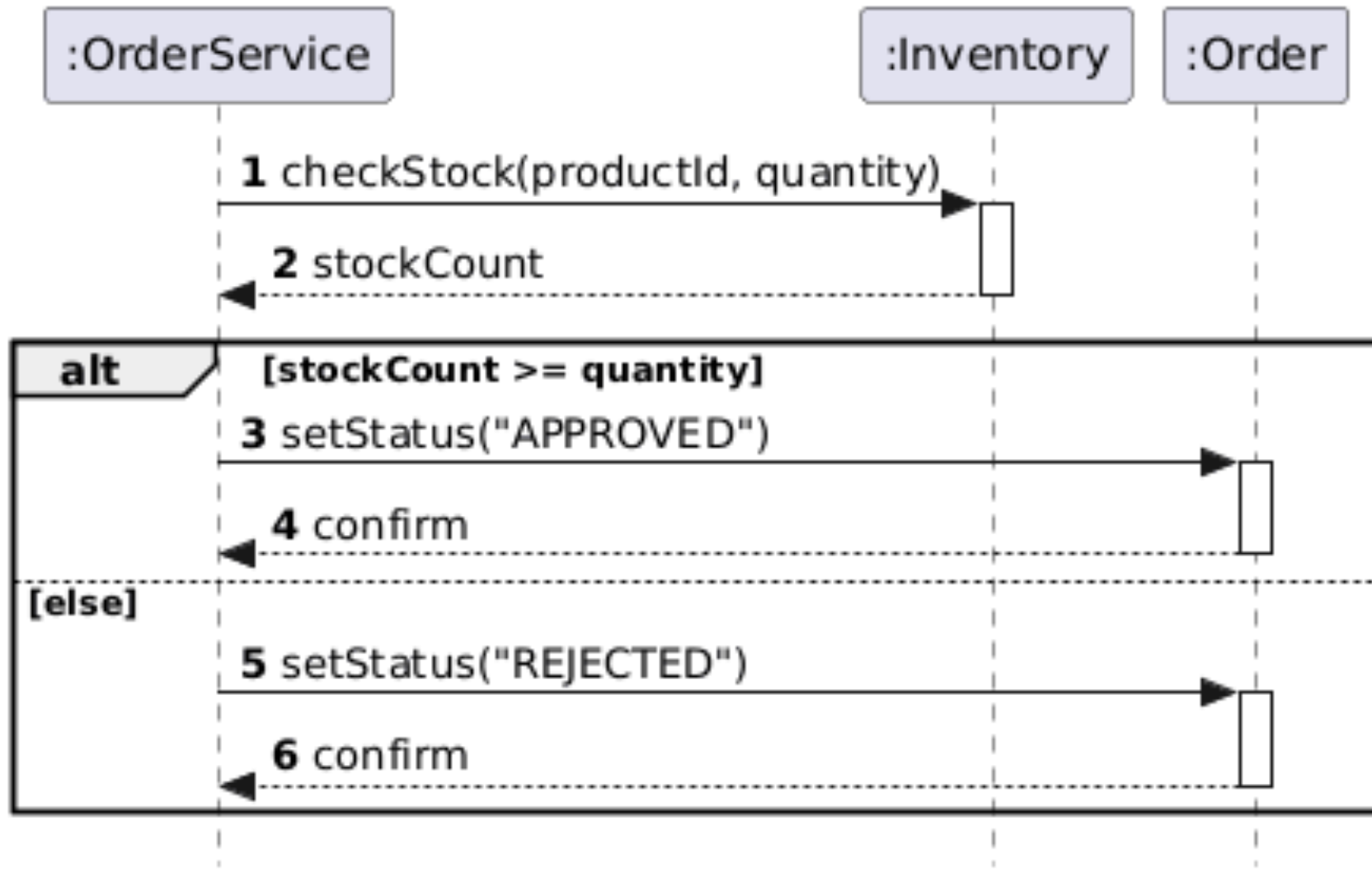
1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram

Combined Fragment (Đoạn kết hợp)

- alt: Alternatives (Rẽ nhánh điều kiện): Combined Fragment alt biểu diễn sự rẽ nhánh loại trừ nhau: chỉ một và đúng một nhánh được thực thi tại một thời điểm, tùy theo điều kiện (guard condition).
- **Tương đương với cấu trúc if-else hoặc switch-case** trong lập trình. Các nhánh được phân cách bằng đường nét đứt ngang bên trong hình chữ nhật combined fragment. Nhánh else (không có điều kiện cụ thể) được ghi là [else].

1. Lược đồ tương tác trong UML



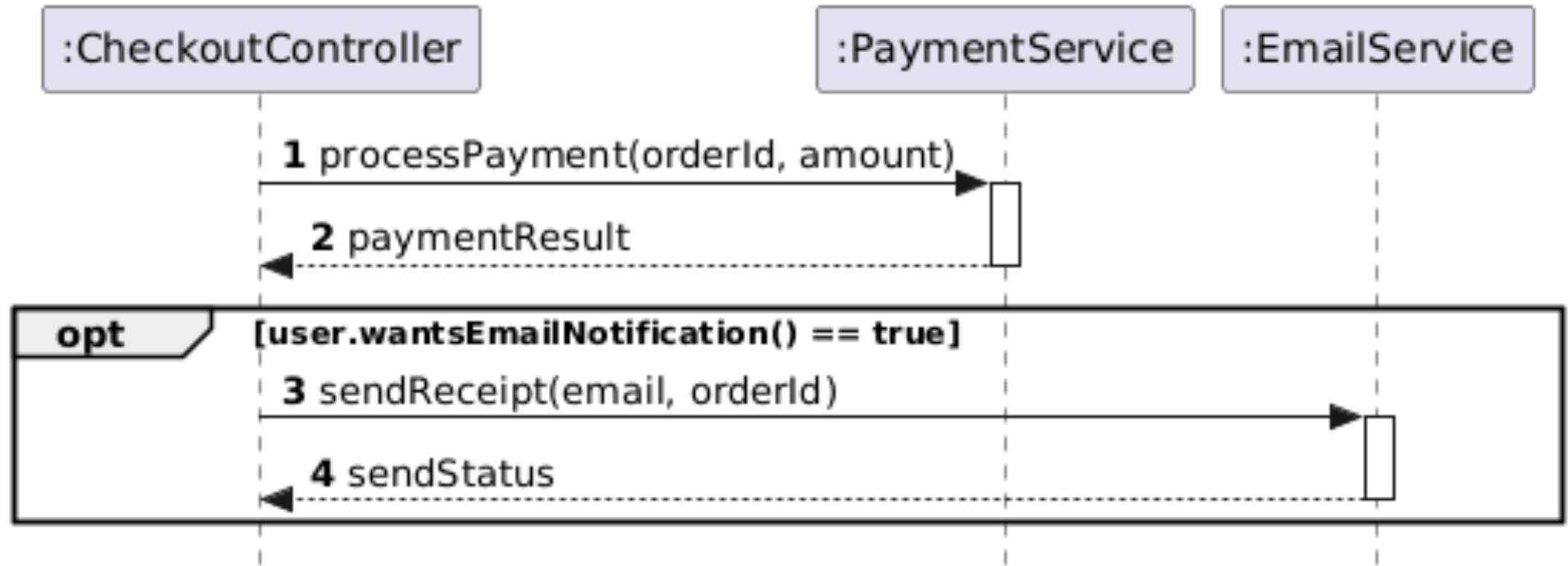
1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram

Combined Fragment (Đoạn kết hợp)

- **opt: Option (Tùy chọn):** Combined Fragment opt biểu diễn một đoạn tương tác chỉ được thực thi khi điều kiện (guard condition) thỏa mãn, và hoàn toàn bị bỏ qua nếu điều kiện không thỏa mãn.
- **Tương đương với cấu trúc if (không có else)** trong lập trình. Khác với alt, opt chỉ có một nhánh duy nhất.

1. Lược đồ tương tác trong UML



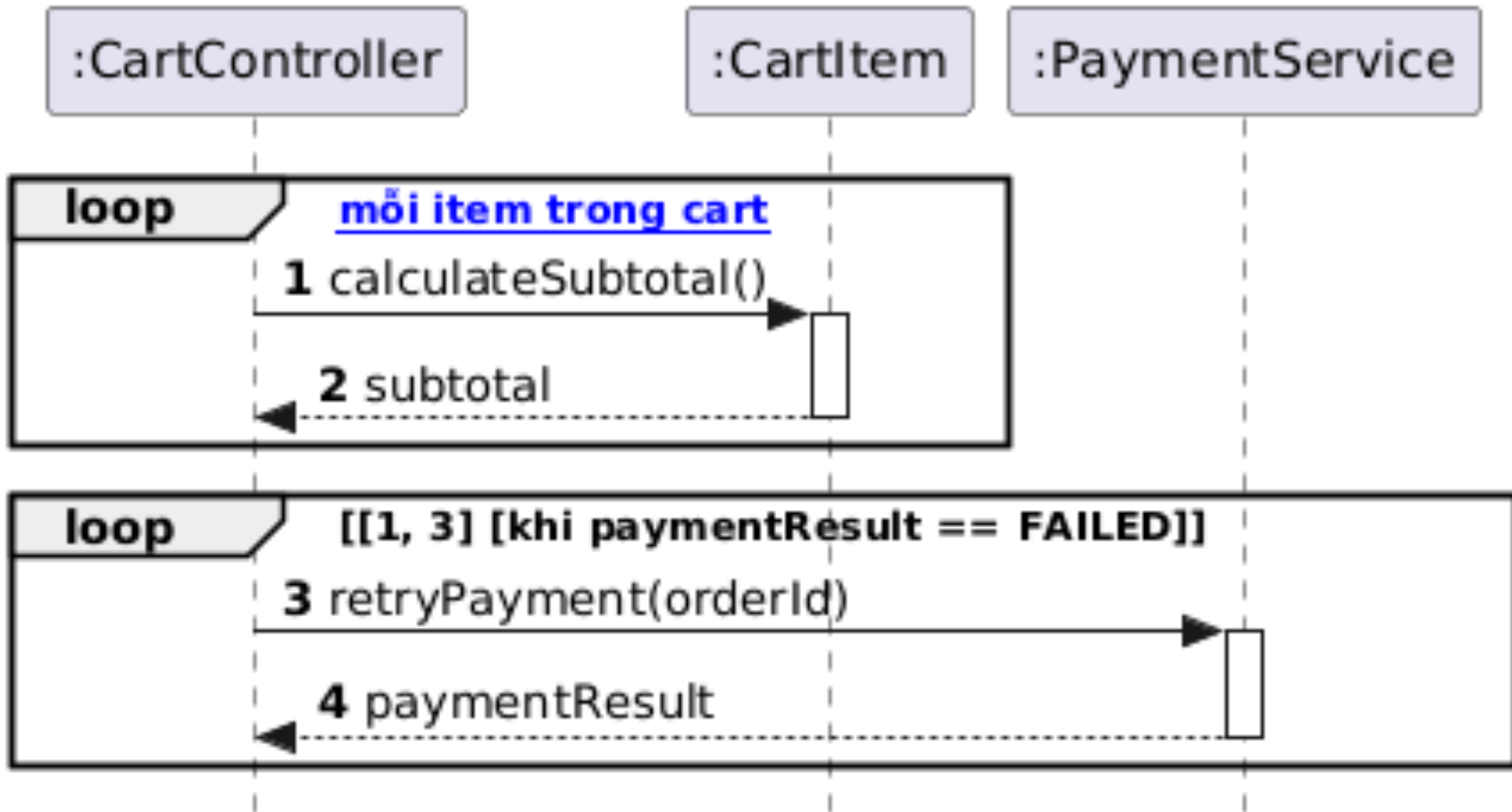
1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram

Combined Fragment (Đoạn kết hợp)

- loop: Loop (Vòng lặp): Combined Fragment loop biểu diễn một đoạn tương tác được thực thi lặp đi lặp lại.
- Điều kiện lặp được ghi trong guard condition, có thể là biểu thức điều kiện hoặc cặp số [min, max] để chỉ định số lần lặp tối thiểu và tối đa.
- **Tương đương với cấu trúc while, for, hoặc do-while trong lập trình.**

1. Lược đồ tương tác trong UML



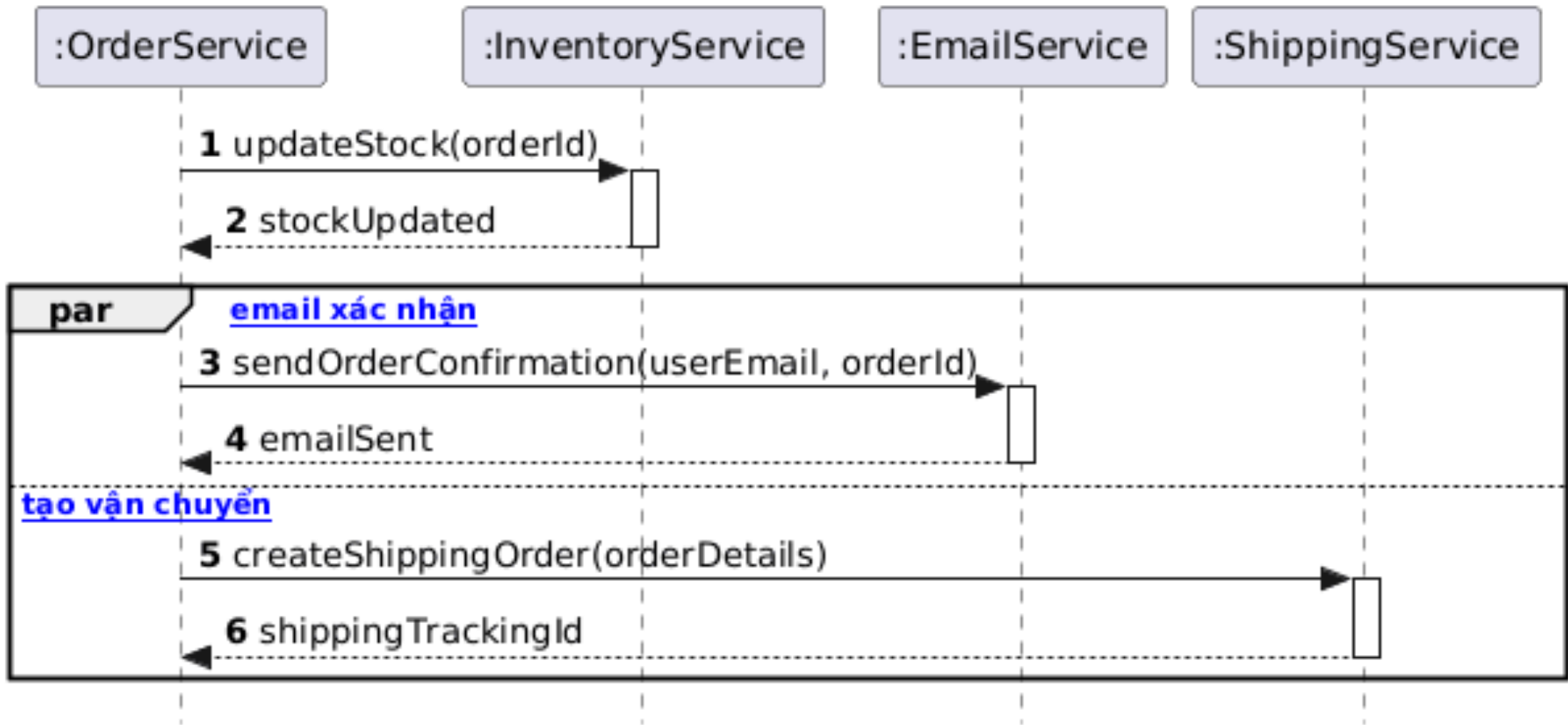
1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram

Combined Fragment (Đoạn kết hợp)

- par: Parallel (Song song): Combined Fragment par biểu diễn các đoạn tương tác được thực thi đồng thời (concurrently) và độc lập với nhau.
- Các nhánh song song được phân cách bằng đường nét đứt.
- Không có thứ tự xác định giữa các thông điệp ở các nhánh khác nhau, chúng có thể xen kẽ theo bất kỳ thứ tự nào.

1. Lược đồ tương tác trong UML



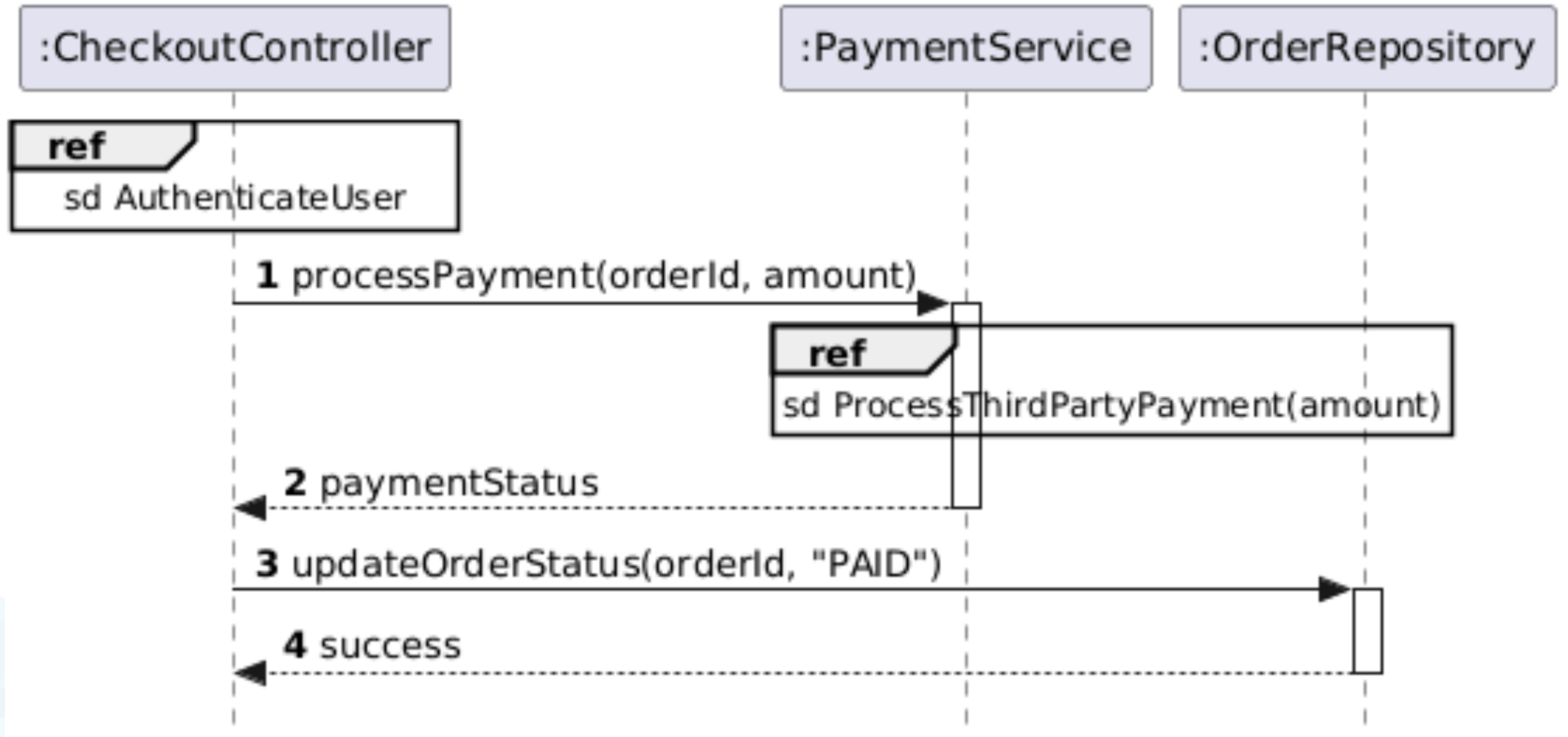
1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram

Combined Fragment (Đoạn kết hợp)

- ref: Reference (Tham chiếu): Combined Fragment ref không biểu diễn logic điều khiển mà là một cơ chế tái sử dụng: **nó tham chiếu đến một Sequence Diagram khác đã được định nghĩa riêng.**
- Thay vì vẽ lại toàn bộ tương tác con vào diagram hiện tại, ta dùng ref kèm tên của diagram con đó. Đây là cách để giữ cho mỗi diagram không quá phức tạp.

1. Lược đồ tương tác trong UML



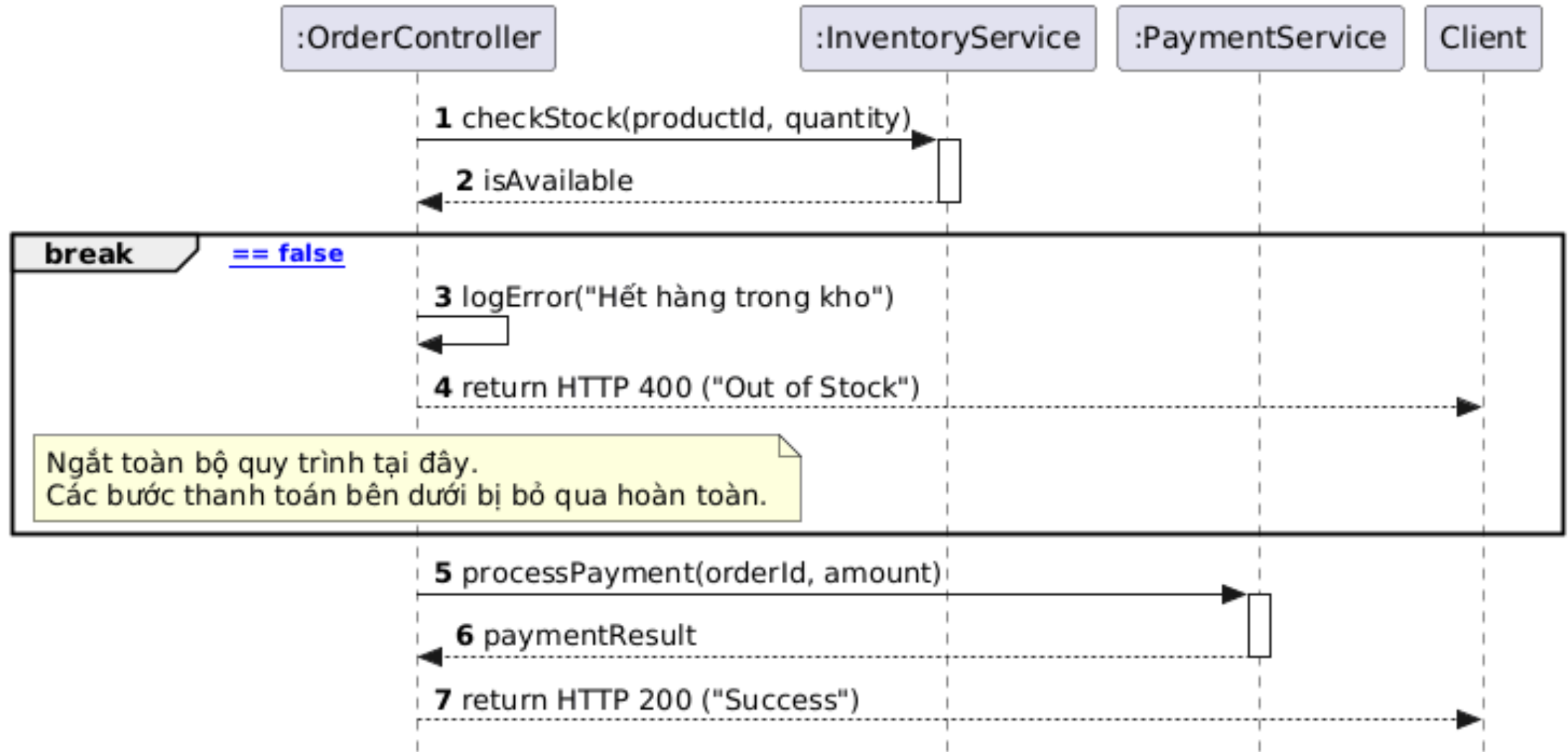
1. Lược đồ tương tác trong UML

1.2.1. Các thành phần của Sequence Diagram

Combined Fragment (Đoạn kết hợp)

- break: Break (Ngắt): Combined Fragment break biểu diễn một điều kiện ngoại lệ (exceptional condition): **nếu điều kiện xảy ra, các thông điệp trong đoạn break được thực thi và toàn bộ tương tác kết thúc sớm (bỏ qua phần còn lại).**
- Tương đương với cơ chế throw exception kết hợp với return sớm trong lập trình.

1. Lược đồ tương tác trong UML



2. Ví dụ minh họa

Ví dụ mô tả chức năng Đăng nhập với luồng thành công, luồng thất bại và tính năng khóa tài khoản sau nhiều lần thất bại liên tiếp.

Kịch bản 1: Đăng nhập thành công (happy path):

- 1) User nhập username và password vào form, bấm nút 'Đăng nhập'.
- 2) loginForm gọi `authenticate(username, password)` đến `authService` [synchronous call - chờ kết quả].
- 3) `authService` gọi `findByUsername(username)` đến `userRepo` để lấy thông tin user từ CSDL.
- 4) `userRepo` thực thi câu lệnh SQL, trả về đối tượng `User` (hoặc null nếu không tìm thấy).

2. Ví dụ minh họa

Ví dụ mô tả chức năng Đăng nhập với luồng thành công, luồng thất bại và tính năng khóa tài khoản sau nhiều lần thất bại liên tiếp.

Kịch bản 1: Đăng nhập thành công (happy path):

5) authService kiểm tra: user tồn tại VÀ mật khẩu hash khớp (checkPassword(inputPwd, storedHash)).

6) authService gọi generateToken(user) để tạo JWT token, trả về AuthResult(success=true, token).

7) loginForm nhận kết quả success, gọi redirectToHome() — điều hướng đến trang chủ.

2. Ví dụ minh họa

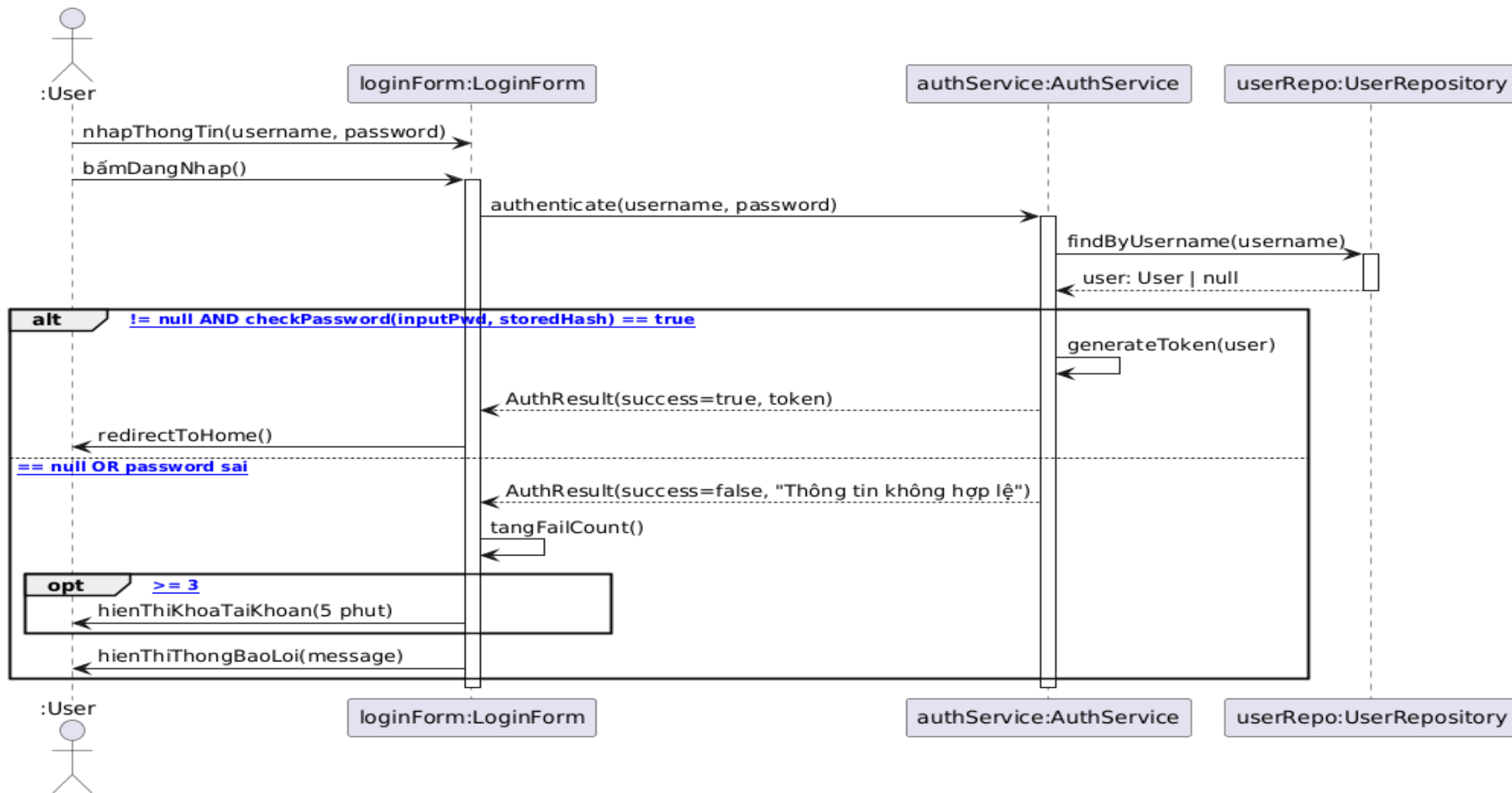
Ví dụ mô tả chức năng Đăng nhập với luồng thành công, luồng thất bại và tính năng khóa tài khoản sau nhiều lần thất bại liên tiếp.

Kịch bản 2: Đăng nhập thất bại (trong nhánh alt [else]):

- 1) authService trả về AuthResult(success=false, message='Tên đăng nhập hoặc mật khẩu không đúng').
- 2) loginForm hiển thị thông báo lỗi, tăng biến đếm failCount lên 1.
- 3) [opt: failCount >= 3] loginForm khóa nút đăng nhập trong 5 phút, hiển thị thông báo khóa.

2. Ví dụ minh họa

sd DangNhap



3. Nguyên tắc vẽ lược đồ tuần tự

- **Một diagram, một kịch bản:** mỗi Sequence Diagram chỉ mô tả một ca sử dụng hoặc một luồng chính + nhánh lỗi của ca đó.
- **Tên thông điệp là tên phương thức thực tế:** placeOrder(cart, userId), không viết 'xử lý đơn hàng'.
- **Luôn có nhánh lỗi:** dùng alt để mô tả ít nhất trường hợp thành công và thất bại.
- **Tách diagram phức tạp:** nếu diagram có trên 15–20 thông điệp, dùng ref để tách luồng con.
- **Thứ tự từ trên xuống dưới:** thông điệp xảy ra trước nằm ở trên, thông điệp sau nằm ở dưới.

3. Nguyên tắc vẽ lược đồ tuần tự

Lỗi cần tránh:

- **Thiếu activation bar**: không biết đối tượng nào đang xử lý tại thời điểm nào.
- **Thiếu return message**: không rõ kết quả trả về là gì.
- **Nhầm opt và alt**: dùng opt khi chỉ có một nhánh; dùng alt khi có hai nhánh loại trừ nhau.
- **Tên thông điệp mơ hồ**: 'gửi dữ liệu', 'xử lý' không đủ thông tin để lập trình viên implement.
- **Diagram quá dài**: nhét tất cả vào một diagram làm mất đi tính rõ ràng — hãy tách nhỏ.
- **Thiếu nhánh lỗi**: chỉ vẽ happy path là thiếu sót nghiêm trọng trong thiết kế.

4. Lớp xử lý

Tiêu chí so sánh	Lớp dữ liệu (Chương 3)	Lớp xử lý (Chương 5)
Câu hỏi trả lời	Dữ liệu được tổ chức như thế nào?	Các đối tượng làm gì và giao tiếp với nhau ra sao?
Trọng tâm	Thuộc tính (attributes), kiểu dữ liệu	Phương thức (methods), luồng gọi, trách nhiệm
Tương đương với	ERD / Database schema / Bảng CSDL	Controller / Service / Repository trong code
Ví dụ với lớp Order	orderId: String; customerId: String; totalAmount: Double; status: OrderStatus	placeOrder(cart, userId); cancelOrder(orderId); calculateTotal()
Quan hệ chính	Association, Aggregation (liên kết dữ liệu)	Dependency, Realization, Generalization (liên kết hành vi)
Đầu ra	ERD → Bảng CSDL	Code skeleton → Lập trình viên implement

4. Lớp xử lý

Kiến trúc phân tầng (Layered Architecture)

- ❑ Kiến trúc phân tầng là một mẫu kiến trúc phần mềm (architectural pattern) trong đó hệ thống được tổ chức thành các tầng (layer) riêng biệt, mỗi tầng có trách nhiệm xác định và chỉ giao tiếp với tầng liền kề.
- ❑ Nguyên tắc cốt lõi: tầng trên có thể gọi tầng dưới, nhưng tầng dưới không được gọi ngược lại tầng trên. Kiến trúc này giúp hệ thống dễ bảo trì, dễ kiểm thử và dễ mở rộng.

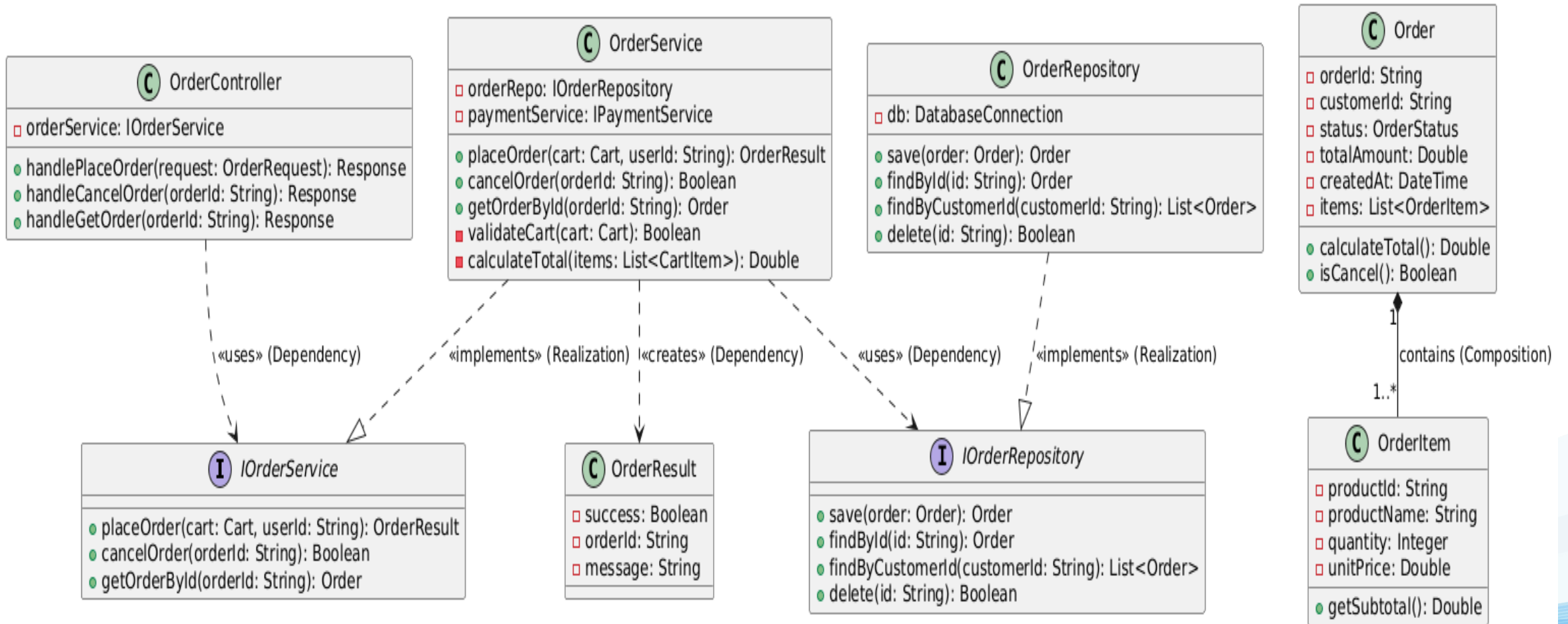
4. Lớp xử lý

Kiến trúc phân tầng (Layered Architecture)

Tầng	Lớp điển hình	Trách nhiệm	Không được làm
Presentation Layer (Tầng trình bày)	Screen, Form, ViewController	Nhận input từ người dùng. Hiển thị kết quả. Gọi Business Layer.	Chứa logic nghiệp vụ. Truy cập trực tiếp database.
Business Layer (Tầng nghiệp vụ)	Service, Manager, UseCase	Xử lý logic nghiệp vụ: validate, tính toán, phối hợp. Gọi Data Access Layer.	Chứa code SQL. Biết về giao diện người dùng.
Data Access Layer (Tầng truy cập dữ liệu)	Repository, DAO	Thực hiện CRUD với database. Trả về Entity/DTO cho Business Layer.	Chứa logic nghiệp vụ. Gọi lên Presentation Layer.
Model / Entity	User, Order, Product, DTO	Đối tượng dữ liệu thuần: chứa dữ liệu, không có logic xử lý phức tạp.	Gọi đến các lớp ở tầng trên. Chứa câu lệnh SQL.

4. Lớp xử lý

Kiến trúc phân tầng (Layered Architecture)



5. SOLID Principles trong lược đồ lớp

- ❑ SOLID Principles (Bộ 5 nguyên lý SOLID) là nền tảng cốt lõi trong thiết kế phần mềm hướng đối tượng (OOD), được đúc kết nhằm giải quyết bài toán khủng hoảng mã nguồn khi hệ thống mở rộng.
- ❑ Bản chất của SOLID là chuyển đổi những đoạn code cứng nhắc, chắp vá thành một cấu trúc linh hoạt bằng cách đẩy mạnh tính đơn nhiệm, tận dụng sự trừu tượng (Interface/Abstract Class) và triệt tiêu sự phụ thuộc trực tiếp giữa các thành phần.
- ❑ Áp dụng SOLID không giúp code chạy nhanh hơn về mặt hiệu năng phần cứng, nhưng giúp hệ thống đạt được “độ bền vững công nghiệp”: dễ dàng cài cắm tính năng mới mà không sợ phá vỡ tính năng cũ, dễ viết Unit Test và giảm tối đa chi phí bảo trì dài hạn.

5. SOLID Principles trong lược đồ lớp

S - Single Responsibility Principle (Đơn nhiệm)

O - Open/Closed Principle (Mở/Đóng)

L - Liskov Substitution Principle (Thay thế Liskov)

I - Interface Segregation Principle (Phân tách Interface)

D - Dependency Inversion Principle (Đảo ngược phụ thuộc)

THỰC HÀNH